CS342 · VISUAL PROGRAMMING

Final Drill Solutions

All the rules, then 8 questions in new forms within the midterm scope

Muharib

01 Solution Q1: constructors

```
A()
B()
C()
A.show
B.show
A()
A.show
```

1. new C(): each constructor calls its parent first, so A() then B() then C().
2. x.show(): the real object is C, which has no show, so B's version runs. B's version calls super.show() first: A.show then B.show.
3. new A(): A has no parent besides Object, so only A(); z.show() is A's version.

02 Solution Q2: casting

```
Dog makes a sound
Cat meows
Animal makes a sound
Dog fetches
Exception in thread "main" java.lang.ClassCastException
```

1. a[0] is a Dog: speak is not overridden so Animal's version runs, but inside it name() runs **Dog's** version because the real object is a Dog. So it prints Dog makes a sound.
2. a[1] is a Cat that overrides speak: Cat meows.
3. (Dog) a[0] is fine because it really is a Dog. (Dog) a[1] is a Cat: ClassCastException; the program stops and End is not printed.

03 Solution Q3: compile or not

#	Answer	Why
a	**Compile error**	An abstract class cannot be instantiated
b	**Compile error**	Bird does not implement fly()
c	**Run-time error**	The downcast compiles, but the object is a String, not an Integer: ClassCastException
d	**Compile error**	The variable type is Animal, which has no fetch
e	**OK**	The real object is a Dog, so the cast is valid
f	**Compile error**	A final class cannot be inherited
g	**Run-time error**	NumberFormatException
h	**Compile error**	Exception already catches everything, so the second catch is unreachable

04 Solution Q4: exceptions

```
ok 10
done 0
not a number
done 1
too big: 70
done 2
ok 5
done 3
total = 30
```

i	Input	What happens
0	"10"	check returns 20, total = 20, prints ok 10
1	"abc"	parseInt throws NumberFormatException, so ok is skipped
2	"70"	70 is above 50, so TooBigException; getMessage returns too big: 70
3	"5"	check returns 10, total = 30

finally prints done i after every iteration, with or without an exception. The catch is inside the loop, so the loop continues.

05 Solution Q5: custom exception

```
class AgeException extends Exception {
    public String toString() { return "Invalid age"; }
}
public class Person {
    private int age;
    public void setAge(int a) throws AgeException {
        if (a < 0 || a > 120) throw new AgeException();
        age = a;
    }
    public static void main(String[] args) {
        Person p = new Person();
        try {
            p.setAge(30);
            System.out.println("Age set");
            p.setAge(150);
            System.out.println("Age set");
        }
        catch (AgeException e) {
            System.out.println(e);
        }
    }
}
```

Output: **Age set** then **Invalid age**. The second "Age set" is skipped because the exception jumps to the catch.

06 Solution Q6: draw the window



1. Up across the whole top, Down across the bottom, Left and Right on the sides.
2. In the middle a panel with GridLayout(2, 2): buttons 1 and 2 on the first row, 3 and 4 on the second, all the same size.

b) BorderLayout. setLayout is not needed because it is the default layout of the JFrame content pane.

07 Solution Q7: write the window

```
import javax.swing.*;
import java.awt.*;
public class Q7 extends JFrame {
    private JLabel title, lName, lId;
    private JTextField tName, tId;
    private JButton bSave, bCancel;
    public Q7() {
        super("Register");
        setSize(320, 170);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        JPanel p = (JPanel) getContentPane();          // BorderLayout by default
        title = new JLabel("New Student");
        p.add(title, BorderLayout.NORTH);
        JPanel center = new JPanel(new GridLayout(2, 2));
        lName = new JLabel("Name:");
        tName = new JTextField(10);
        lId   = new JLabel("ID:");
        tId   = new JTextField(10);
        center.add(lName);  center.add(tName);
        center.add(lId);    center.add(tId);
        p.add(center, BorderLayout.CENTER);
        JPanel south = new JPanel();                   // FlowLayout by default
        bSave   = new JButton("Save");
        bCancel = new JButton("Cancel");
        south.add(bSave);
        south.add(bCancel);
        p.add(south, BorderLayout.SOUTH);
    }
    public static void main(String[] args) {
        Q7 f = new Q7();
        f.setVisible(true);
```

```
    }
}
```

The add order in the GridLayout matters: Name then its field, then ID then its field, because the cells fill row by row.

08 Solution Q8: MCQ

#	Answer	Reason
1	**b**	The JFrame content pane defaults to BorderLayout
2	**a**	JPanel defaults to FlowLayout
3	**a**	The first number is rows, the second columns
4	**c**	Swing is in javax.swing; AWT is in java.awt
5	**d**	Shrinks the window to the smallest size that shows every component
6	**b**	All Swing components are lightweight except those four